Project Proposal

AL-2001 Programming for Artificial Intelligence (Laboratory) · Fall 2026 · Semester Lab Project

FAST — National University of Computer and Emerging Sciences, Karachi Campus

Group Information

	Name	Roll Number	Section	Role in the project
Leader	Taha Mahmood	25K-0124	BAI-3A	Member A: data layer, OOP core and console menu
Member 2	Syed Shahamat Shakeel	25K-0143	BAI-3A	Member B: text processing, analysis and models

Project Title	SolarSense: Home Solar System Planner
Specialisation Track	Track A — Text / NLP ☑
GitHub Repository URL	TahaMahmood-25/AL-2001-PAI-Project-SolarSense
Date of Submission	19 October 2026

1. Project Title and Summary

SolarSense: Home Solar System Planner

SolarSense is a Python console app that helps a household work out what size of solar system it needs. The user types their appliances and daily hours in plain words, for example "2 ACs 8 hours, 4 fans, fridge". The program cleans the text, finds each appliance's power use and calculates the home's daily electricity need. Using real sunshine data for the user's city from a free weather service, it recommends a system size and shows the expected solar output for every month. Two machine learning models are

compared at predicting daily sunshine from weather, and the better one is used to estimate tomorrow's solar output. The user gets a clear, evidence-based estimate before talking to an installer.

2. Problem Statement and Motivation

In Pakistan, high electricity bills and frequent load-shedding have led many households to consider solar power. The first question they face is simple but hard to answer: how big should the system be? A family with two air conditioners, several fans and a fridge usually decides by asking an installer, copying a neighbour's system, or guessing from last month's bill. None of these methods uses the home's own appliances or the actual sunshine in its city, and sunshine changes a great deal between summer and winter, and between a clear and a cloudy month. A system that is too small leaves the household paying the grid, while one that is too large wastes money. Ordinary homeowners have no easy tool for working this out themselves. A free tool that turns a list of appliances and real local sunshine data into a clear size estimate would help households ask better questions before they spend money.

3. Proposed Solution

The user starts the console app and chooses their city. They then type their appliances and daily hours in plain words, for example "2 ACs 8 hours, 4 fans 12 hours, fridge". The program cleans this text, matches each phrase to an appliance in its wattage table, and calculates the home's daily electricity use in kWh. It then loads a year of hourly sunshine data for the chosen city from a free weather API (or from its saved copy when offline) and works out how much energy one kilowatt of panels would produce in each month. From these two results it recommends a system size in kW, converts it into an approximate number of panels, and prints a table comparing expected monthly solar output with the home's use. Finally, the better of two trained models estimates tomorrow's sunshine from the weather forecast. Results are shown as charts and saved as a report.

Main components

- **Appliance parser** — cleans the typed text and matches each phrase to an appliance in the wattage table.
- **Energy calculator** — turns appliances and hours into the home's daily kWh.
- **Weather data client** — fetches sunshine and weather data from the API, checks the response and keeps a local cache.

- **Solar estimator** — uses NumPy to turn hourly sunshine into monthly output and a recommended system size.
- **Prediction module** — trains and compares two models that predict daily sunshine from weather.
- **Report and charts module** — draws the charts and saves the results.
- **Console menu (main)** — ties everything together and validates all user input.

3.1 Planned Class Design

- `Appliance` (abstract base class)
  - Attributes: name, watts, hours per day (kept private, read through properties)
  - Methods: `daily_kwh()` (abstract), `describe()`
  - `ConstantLoadAppliance` extends `Appliance` — for items such as lights and fans that draw power for all the hours they are on; overrides `daily_kwh()`.
  - `CyclingAppliance` extends `Appliance` — for items such as fridges and ACs whose compressor switches on and off; adds a duty-cycle factor and overrides `daily_kwh()`.
- `Household`
  - Attributes: city, list of `Appliance` objects
  - Methods: `add_appliance()`, `total_daily_kwh()` (adds up `daily_kwh()` of every appliance, whatever its type), `summary()`
- `WeatherClient`
  - Attributes: latitude and longitude, cache folder
  - Methods: `fetch_history()`, `fetch_forecast()` (check the status code, save to the cache, read the cache when the network fails)
- `SolarEstimator`
  - Attributes: sunshine data (NumPy array), system efficiency factor
  - Methods: `monthly_output()`, `recommend_size()`
- `SolarPredictor`
  - Attributes: two models, train/test data
  - Methods: `train()`, `evaluate()`, `predict_tomorrow()`
- `WeatherDataError` — custom exception raised when no live or cached data is available.

4. Scope

In scope — features we commit to delivering

#	Feature	Why it is essential
1	Appliances typed in plain words are cleaned and matched to a wattage table	This is the main input and our text-processing (Track A) component
2	Daily electricity use is calculated from appliances and hours	Every recommendation depends on this number
3	Real sunshine data is fetched from a public API, checked, and cached for offline use	Makes the result specific to the user's city and keeps the program working without internet
4	Recommended system size and expected monthly solar output compared with household use, with charts	This is the main result the user sees
5	Two machine learning models predict daily sunshine, are compared on unseen months, and the better one estimates tomorrow's output	Required honest model evaluation; also gives the user a forward-looking estimate

Out of scope — deliberately excluded

Graphical or web interface; payback or cost analysis; battery, inverter and wiring design; roof shading and panel angle; net-metering and government rules; live panel or electricity prices; deep learning; a database server; deployment. SolarSense gives estimates for planning conversations. It is not professional engineering advice. As a group of two, we have deliberately kept the scope smaller than a group of three would.

5. Data Source

Source name and URL	Open-Meteo Historical Weather API (https://archive-api.open-meteo.com/v1/archive) and Forecast API (https://api.open-meteo.com/v1/forecast); project site: https://open-meteo.com. Plus a small appliance wattage table (CSV) that we compile ourselves.
Type	Public web API ☑, Self-collected / generated ☑ (appliance table)
Access requirements	No key needed ☑
Approximate size	About 8,760 hourly rows per city per year. We tested Karachi for 2025: 8,760 hours received, no missing sunshine values. Appliance table: roughly 40–60 rows.
Licence / terms of use	Open-Meteo offers its weather data free for non-commercial use with attribution; we will confirm the exact terms on its website before submission and credit it in the README and report. Appliance wattages are typical published values; sources will be listed in the report [to be completed].
Offline fallback	Every API response is saved as a JSON file in `data/raw/` for each city and date range. The program checks the saved copy when the request fails (no network, error status or timeout) and prints a clear message. If neither live nor saved data exists, `WeatherDataError` is raised and handled by the menu. A year of data for a short list of major Pakistani cities is saved in advance.

6. Technologies and Lab Coverage

Lab	Concept to be applied	How our project will use it
01–03	Core Python, control flow, data structures	Validated menu input and type conversion for hours and counts; loops for the menu; a dictionary for the appliance table, a set for unique appliances and stop words, a tuple for each city's coordinates, lists for the household's appliances
04	Functions, lambda, map / filter	Functions with parameters, defaults and docstrings (for example `clean_text(text, remove_stopwords=True)`); `sorted(key=lambda ...)` to rank appliance matches; `map` and `filter` to convert and check typed values
05	Classes, inheritance, encapsulation, polymorphism	Abstract base class `Appliance` with two subclasses overriding `daily_kwh()`; `Household` sums them polymorphically; private attributes with properties
06	Files (TXT / CSV / JSON), exceptions, web API	Read the CSV appliance table; JSON cache of API data; TXT report output; specific `except` blocks and the custom `WeatherDataError`; `requests` with status-code check and offline cache
07	NumPy arrays, broadcasting, vectorisation	Hourly sunshine reshaped to a days × 24 array; boolean mask for daylight hours; broadcasting to apply system size and efficiency; sums and means along an axis for daily and monthly totals, with no element-by-element loops
08	Pandas — loading, cleaning, grouping	DataFrame of hourly weather; handle missing values; filter by date; group by month and summarise

Lab	Concept to be applied	How our project will use it
09	Matplotlib / Seaborn visualisations	Three labelled charts: monthly solar output vs household use; sunshine vs cloud cover; predicted vs actual sunshine for the models. Each is interpreted in the report
10–11	Machine learning, model comparison	Train/test split by time; Linear Regression vs Random Forest predicting daily sunshine from cloud cover, temperature and similar weather features; compared with MAE and R^2 on unseen months
12	Track A — NLP text features	Our own code lowercases, removes punctuation and normalises variants ("a/c", "air conditioner" to "ac"); TF-IDF turns text into features; cosine similarity matches each phrase to the appliance table
14	Documentation, modularity, ethical considerations	Modular `src/` layout, docstrings, README, pinned `requirements.txt`; ethics: results are estimates, not professional advice; no personal data stored; data source credited; AI assistance cited

7. Division of Work

Member	Modules / components owned	Approximate share
Taha Mahmood (Leader)	Repository setup and README; `Appliance`, `Household` classes; `WeatherClient`, caching and file handling; `WeatherDataError`; console menu and input validation	50%
Syed Shahamat Shakeel	Text cleaning and TF-IDF matching; `SolarEstimator` (NumPy); Pandas analysis; charts; `SolarPredictor` (two models and evaluation)	50%

Both members will commit from their own GitHub accounts from the first week, and each member will be able to explain their own modules in the viva.

8. Milestones and Timeline

#	Milestone — what will be demonstrably working	Target date	Owner
1	Repository created with both members' commits; appliance table loads from CSV; weather client fetches a year of data and saves it to the cache; menu skeleton runs	Week 1	Taha
2	A typed sentence is cleaned and matched to appliances; `Household` prints the daily kWh	Week 2	Shahamat (matching), Taha (classes)
3	NumPy estimator prints a recommended system size and monthly output; Pandas summary works	Week 3	Shahamat
4	Two models trained and compared on unseen months; three charts saved; tomorrow's estimate shown in the menu	Week 4	Shahamat (models), Taha (menu)

#	Milestone — what will be demonstrably working	Target date	Owner
5	Final integration, documentation and repository cleanup (clean-clone test on another machine)	Week 5	Taha and Shahamat

9. Expected Outcome and Risks

Examiner will run one command and see a clear menu. After typing a sentence such as "2 ACs 8 hours, 4 fans, fridge", they will see the appliances the program recognised, the home's daily electricity use, a recommended solar system size in kW, and a month-by-month table of expected solar output. They can ask for tomorrow's estimate and view the saved charts. Unplugging the internet will not break the program, because it falls back to saved data. In the report we will present the comparison of two models on months they did not train on (using MAE and R^2), three interpreted charts, and test results for bad input.

Risk 1 — The API is unreachable or limits our requests. We will download and cache a full year of data for each supported city early, so the program runs from saved files. If the network fails, the menu shows a clear message and uses the cache.

Risk 2 — Typed appliances are not recognised or the wattage values are unreliable. If a phrase's similarity score is too low, the program will ask the user to choose from a numbered list or enter the watts directly. We will add common spelling variants to the table and label all results as estimates.

AI assistance: We used an AI assistant (Claude) to brainstorm project ideas and to help structure this proposal. It will be cited in the final report together with any other external help.

For office use — Lab Instructor

Proposal status	Approved ☐ Approved with changes ☐ Revision required ☐
Comments	
Signature and date	